

第六届计图人工智能挑战赛

赛题二点云降噪方法与代码说明

队伍：NKAI 排名：031 A 榜成绩：79.23

一、队伍信息

队伍名称	NKAI
A 榜排名	31
A 榜成绩	79.23 / 67.42 / 91.04 (Total / CD / P2S)
团队成员	李佳璞, 电话/微信: 18883873159
团队成员	刘迪乘, 电话/微信: 18570499315
团队成员	曲恒睿, 电话/微信: 17606383128
提交文件名	contest2_NKAI_031.zip

二、我们的方法

我们把赛题中的降噪任务看作带索引约束的点位移估计：输入和输出都保留 50,000 个点，输出中的第 i 个点只对应输入中的第 i 个点。这样既能修正局部噪声，又不会在拼接过程中改变点数或打乱顺序。

我们的主体网络由四个连续的去噪模块组成。每个模块先用动态边卷积提取局部邻域特征，再用状态空间模块补充长距离信息，最后预测该阶段的点位移。后一模块接收前一模块修正后的点云，因此网络可以逐步处理从明显离群点到细小表面抖动的不同误差。

状态空间部分由我们直接用 Jittor 实现。选择性扫描的前向和反向均通过 `jt.code` 编写 CUDA 算子；训练和推理不调用 PyTorch、Triton、`mamba_ssm` 或 `causal_conv1d`。为了控制显存，我们每隔 16 个序列位置保存一次状态，反向计算时从这些状态继续。提交模型使用 16 个状态维度和 `float32`。

单一模型很难同时兼顾桌类的大平面、沙发类的软边界以及低样本类别。我们因此从同一个基础模型出发，按数据形态训练密度分支和类别专家。各分支共享网络结构，只改变训练子集、噪声范围和密度感知损失权重。推理时，我们再用一个只依赖含噪点云的参考分支判断专家位移是否过大。这样做不需要测试集真值，也不会读取测试集 `clean` 点云或 `mesh`。

参考分支采用 Jittor 原生 StraightPCF。我们先训练速度模块，再训练级联速度模块和完整点云滤波模块。测试时执行两次滤波，并根据第一遍输出的均值、方差和局部变异系数确定第二遍强度。这个分支不负责替代主体网络，而是给专家输出提供尺度参照和异常回退。

三、数据准备与训练

从 mesh 构建训练点云

我们的训练输入来自原始 ShapeNet mesh：

```
<TRAIN_ROOT>/shapenet/<synset>/<model>/models/model_normalized.obj
```

我们在每个三角面上按面积分配采样概率，确定性地采样 50,000 个表面点。数据目录若带有 `model_normalized.obj.cache.npz`，我们读取其中的顶点和面，以保持 mesh 解析顺序；没有缓存时，我们使用代码包中的标准库 OBJ 解析器。采样随机数由公开 key 派生，因此重复运行会得到同一份 clean 点云。

训练时，我们在线加入 Gaussian 或 Laplace 噪声、2% 离群点以及尺度变化，再从 50K 点云中抽取局部 patch。基础训练覆盖 15,733 个公开训练 mesh；类别训练使用包内固定 key list，列表只保存 `shapenet/<synset>/<model>` 标识，不包含坐标、标签、预测或权重。

主体网络训练

我们从随机初始化训练基础模型 100 个 epoch。单卡 micro-batch 为 4，累计 8 次梯度后更新一次参数，等效 batch 为 32。后续分支使用 micro-batch 4、累计 4 次梯度，等效 batch 为 16。共同参数为 `patch_size=1000`、`patch_ratio=1.2`、四个去噪模块、`frame_knn=32` 和梯度裁剪 1.0。

下表按用途列出流水线中的训练任务。表中的“样本数”表示 mesh 数乘以每个 mesh 的采样次数， λ_d 是密度感知 Chamfer 项的权重。

用途	轮数	样本数	学习率	λ_d	随机种子
基础模型	100	15733×4	10^{-4}	0	2020
类别均衡训练	3	650×4	10^{-5}	0	20260814
密度分支	2	3000×1	5×10^{-6}	0.5	20261303
桌类初始分支	2	1200×2	5×10^{-6}	0	20260903
桌类密度训练	2	1200×1	2×10^{-6}	0.5	20261411
桌类强去噪训练	2	1200×1	10^{-6}	1.0	20261511
沙发强去噪训练	2	1000×1	2×10^{-6}	1.0	20262011
飞机专家	2	1000×4	10^{-6}	0.5	20262701
桌类专家	2	1200×4	10^{-6}	1.0	20262702
沙发专家	2	1000×4	10^{-6}	1.0	20262703
低样本类别专家	2	1885×4	10^{-6}	0.5	20262731

飞机、桌类和沙发专家使用纯 Laplace 噪声。三个区间分别为 $[0.0124896924, 0.0146169608]$ 、 $[0.0090486185, 0.0129036043]$ 和 $[0.0083744542, 0.0113465701]$ 。七个低样本类别各自使用独立区间，具体数值已经写入 `configs/` 下的类别噪声配置，流水线会自动读取。

我们的训练关系可以概括为：

基础模型

```
`-- 类别均衡训练
    |-- 密度分支
    |   |-- 沙发强去噪训练 -- 沙发专家
    |   |-- 飞机专家
    |   `-- 低样本类别专家
    `-- 桌类初始分支 -- 桌类密度训练
        `-- 桌类强去噪训练 -- 桌类专家
```

参考分支训练

StraightPCF 的三个训练阶段依次为速度模块、级联速度模块和完整滤波模块，默认轮数为 3、30、40。每个 epoch 使用 10,000 个样本，batch size 为 32，patch size 为 1,000，Adam 学习率为 10^{-4} 。我们从 mesh 采样 32,768 个表面点，并保留最多 1,024 个原始顶点；训练噪声为 $[0.008, 0.014]$ 的 Laplace 分布，离群点比例为 2%，离群尺度为 3。

四、推理和结果融合

50K 点云推理

测试数据目录为：

```
<TEST_ROOT>/shapenet/<synset>/<model>/noisy.npy
```

我们先在 50K 点云上选择 150 个最远点采样中心，再为每个中心建立 2,000 点 patch。主体网络分别处理这些 patch；同一个原始点若出现在多个 patch 中，我们选择归一化中心距离最小的预测，因此每个输入索引只保留一个位移。该过程不会重新采样输出点。

参考分支执行两遍 StraightPCF。第一遍估计噪声尺度，第二遍按尺度调整滤波强度，随后以 $\gamma = 0.5$ 融合两遍残差。参考输出与主体输出都保持输入索引。

仅含噪输入的位移约束

设含噪点为 $\mathbf{x}_i^n$ ，参考分支输出为 $\mathbf{x}_i^r$ ，类别专家输出为 $\mathbf{x}_i^e$ 。我们计算

$$r_i = \frac{\|\mathbf{x}_i^e - \mathbf{x}_i^n\|_2}{\max(\|\mathbf{x}_i^r - \mathbf{x}_i^n\|_2, \text{tiny}_{64})}, \quad q = Q_{0.95}(\{r_i\}).$$

当 $q \leq 4.0$ 时，我们采用

$$\mathbf{x}^{out} = \text{float32}(0.25 \mathbf{x}^r + 0.75 \mathbf{x}^e);$$

否则使用参考分支输出。这个判断只比较三份由 noisy 输入得到的点云，不使用 clean 点云、标签或官方指标。

按类别使用模型

我们把提交时的模型选择直接写成固定类别映射，没有在测试集上搜索阈值。飞机、沙发和七个低样本类别使用相应专家与上面的位移约束；五个对密度更敏感类别使用密度分支；其余类别使用类别均衡模型。

桌类同时保留桌类强去噪输出 $\mathbf{x}^b$ 和经过位移约束的桌类专家输出 $\mathbf{x}^t$ 。我们使用

$$\mathbf{x}^{table} = \text{float32}\left(\mathbf{x}^b + 1.25(\mathbf{x}^t - \mathbf{x}^b)\right).$$

全部类别列表都随代码提交。我们在 `assemble.py` 中按 key 所属类别读取相应列表并完成组合，不使用测试点云以外的信息。

结果格式

我们把每个结果保存为 `finite float32(50000,3)`：

```
shapenet/<synset>/<model>/denoised.npy
```

打包时，我们按 key 排序并检查成员唯一性、路径安全、数组形状、数据类型和有限值。输出 ZIP 不包含目录穿越项或多余文件。

五、环境与代码结构

运行环境

我们使用的检查环境如下：

- Ubuntu 22.04;
- NVIDIA RTX 4090 24GB, compute capability 8.9;
- CUDA Toolkit 12.4, cuDNN 8.9.7;
- Python 3.10.14;
- Jittor 1.3.11.0, NumPy 1.24.4。

推荐直接创建随包环境：

```
conda env create -f environment.yaml
conda activate <ENVIRONMENT_NAME>
```

若机器已经安装兼容的 CUDA 和 cuDNN，也可以只安装正式运行依赖：

```
pip install -r requirements.txt
```

我们把依赖分成两层：`requirements.txt` 只列 Jittor 和 NumPy；`environment.yaml` 另外固定 CUDA Toolkit、cuDNN 及赛事检查环境中的常用科学计算包。正式训练和推理只导入 Jittor 与 NumPy，PyTorch、Triton、SciPy、trimesh、OmegaConf 和 tqdm 均不在运行路径中。

Conda CUDA 把 `libcudart.so` 放在环境的 `lib/` 中，而 Jittor 1.3.11 默认检查 `lib64/`。我们在 `main.py` 中检测当前 Conda 环境，并在需要时建立 `lib64->lib` 兼容链接。

提交包

```
contest2_NKAI_031.zip
|-- code/
|   |-- main.py
|   |-- train.py
|   |-- train_rot.py
|   |-- infer.py
|   |-- generate_rot.py
|   |-- assemble.py
|   |-- package_results.py
|   |-- rot_jittor/
|   |-- configs/
|   |-- tools/
|   `-- tests/
|-- requirements.txt
|-- environment.yaml
`-- 提交说明文档.pdf
```

我们没有在代码包中放入权重、训练点云、测试预测、mesh、日志或缓存。所有 checkpoint 都在指定工作目录中从随机初始化产生。

六、复现步骤

检查人员可以自行选择数据和输出目录。进入 `code/` 后，将下面三个变量改成机器上的实际绝对路径；其中 `NKAI_WORK_ROOT` 需要可写，并留出训练数据、checkpoint 和预测结果所需空间。

```
cd code
export NKAI_TRAIN_ROOT="/path/to/track2_train"
export NKAI_TEST_ROOT="/path/to/track2_test"
export NKAI_WORK_ROOT="/path/to/work_directory"
```

先检查 Python、Jittor、CUDA 和自定义算子：

```
python main.py doctor
python main.py doctor --deep
```

查看流水线将要执行的命令：

```
python main.py pipeline \  
  --config configs/reproduce_full.json \  
  --log-dir "$NKAI_WORK_ROOT/logs" \  
  --dry-run
```

确认数据路径后执行训练、推理、组合和打包：

```
python main.py pipeline \  
  --config configs/reproduce_full.json \  
  --log-dir "$NKAI_WORK_ROOT/logs"
```

我们为每个耗时步骤指定了完成 checkpoint 或 manifest。重复执行同一命令时，流水线会跳过已经完成的步骤；如果需要重跑某一步，删除该步对应的完成文件即可。StraightPCF 训练支持从最近的训练状态继续。

若只需重新打包已经生成的预测，可以运行：

```
python main.py package-results -- \  
  --root <PREDICTION_ROOT> \  
  --key-list <TEST_KEY_LIST> \  
  --zip <RESULT_ZIP> \  
  --audit <AUDIT_JSON> \  
  --expected-count 200 \  
  --expected-points 50000
```

七、运行验证

我们在 Ubuntu 22.04、RTX 4090、CUDA 12.4.99、cuDNN 8.9.7、Python 3.10.14 和 Jittor 1.3.11.0 环境中解压代码并完成了以下检查：

- 编译全部 Python 文件，并运行源码策略和结果融合单元测试；
- 编译选择性扫描 CUDA 算子，执行前向、反向和梯度检查；
- 从随机初始化更新主体网络参数，并检查密度感知损失和梯度累计；
- 依次训练 StraightPCF 的三个模块，保存、载入并继续训练；
- 对真实 50K 点云运行主体网络和两遍参考滤波；
- 组合单个测试 key，并生成带 CRC 和成员清单的结果 ZIP；
- 展开整条流水线，核对数据、训练、推理和打包命令之间的路径。

主体网络的 50K 实测输出为 finite float32，形状为 (50000,3)，没有触发 patch 回退；参考分支输出同样满足点数、类型和有限值要求。我们的源码审计还会检查禁止后缀、私有路径、密钥模式和禁止运行时依赖。

不同 GPU 上的动态 KNN 临界近邻和 CUDA reduction 次序可能产生很小的浮点差异，因此重新训练所得权重不要求逐字节相同。我们固定了训练 key、随机种子、噪声区间、网络结构、优化参数、类别映射和结果格式；提交结果仍需通过 shape、dtype、index、finite 和 ZIP 成员检查。

八、来源与许可证

我们参考了 3DMambaIPF、Mamba、IterativePFN 和 StraightPCF 的公开论文与实现，并使用 Jittor 完成正式训练和推理。来源说明、许可证文本和我们重写的模块边界见 `code/LICENSES.md` 与 `code/NOTICE`。